Building an Organizational Chart Inside a Production HR Platform

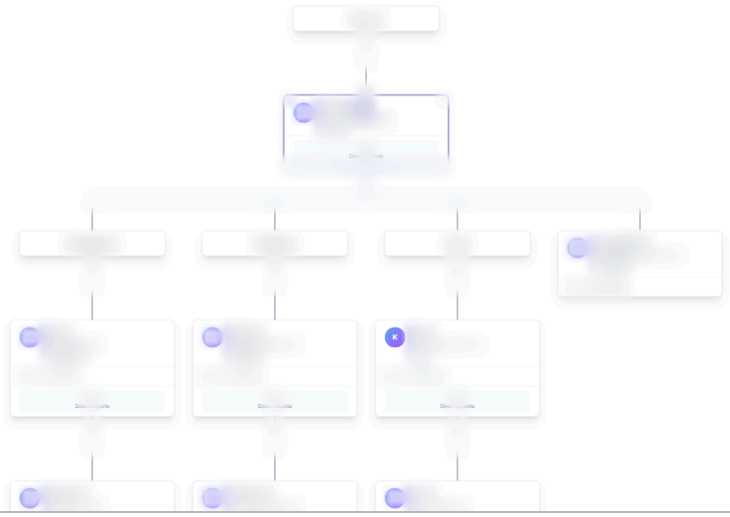
A record of the work, the decisions behind it, the mistakes learned from, and the reasoning that changed along the way.

Author
Gabriel Marcel Danandjaja

Role
Backend Developer Intern

Organisation
UIC Asian Computer Services, Singapore

Period covered
2 to 24 September 2026 (Weeks 1 to 4)



The delivered feature. Department entities and employees are resolved into a single tree by one recursive SQL query, with the signed-in user highlighted and reporting lines derived from data rather than maintained by hand.

External copy: names in screenshots are obscured and internal identifiers replaced with generic equivalents.

01

Overview

This journal covers the first four weeks of a three and a half month internship at UIC Asian Computer Services, a Singapore SME. I joined the backend team on the company's flagship product, an AI driven HR platform intended to cover recruiting, interviewing, onboarding and payroll in one system. My assignment was the organizational chart: a feature that did not exist at all when I arrived.

What follows is deliberately not a list of achievements. It records the reasoning behind each decision, the approaches abandoned and why, what other people's mistakes taught me, and the points where my own understanding turned out to be incomplete, including a significant architectural reversal in week three.

The through line. The single most useful thing I learned in four weeks was that the correct place for a piece of logic is not a matter of taste. It is decided by where the data already lives and what the access pattern actually is. I built the same feature twice, in two different layers, and only the second version was right. The reasoning behind that reversal is in Section 4.

4
Repositories touched across an eight service architecture

3
Recursive CTE query systems designed, built and tested

2
Architectural layers the feature was built in, then rebuilt across

0
Merge conflicts across three engineers on one feature

Where the feature sits

The platform is a multi tenant HR system built on Microsoft Fabric. It grew out of the company's earlier payroll and staffing products, and it is the largest project in the company. The organizational chart is a core navigation surface: it answers who reports to whom, who sits in which department, and where a given employee sits in the structure, including budgeted positions that have not been filled yet.

Dimension	*Detail*
Team	Nine people: three lead developers, three interns joining, three interns leaving. The org chart is owned by three of us.
My slice	Backend and data layer. Query design, tree construction, schema and migrations, orchestration, API contract.
Shared with	A database engineer on the table design and data layer; a frontend engineer on the rendering and the agentic path in Python.
Primary stack	ASP.NET Core on .NET 10, C#, T-SQL on Microsoft Fabric SQL, Python with LangChain, Next.js and TypeScript, Docker Compose, Azure DevOps.
Supporting stack	RabbitMQ, Redis, Eventhouse RAG, Spark, Liquibase, Bicep, Terraform, Azure Container Apps.
Working conditions	A flagship product under sustained schedule pressure, with delivery dates often set without engineering input. Scope had to be negotiated continuously.

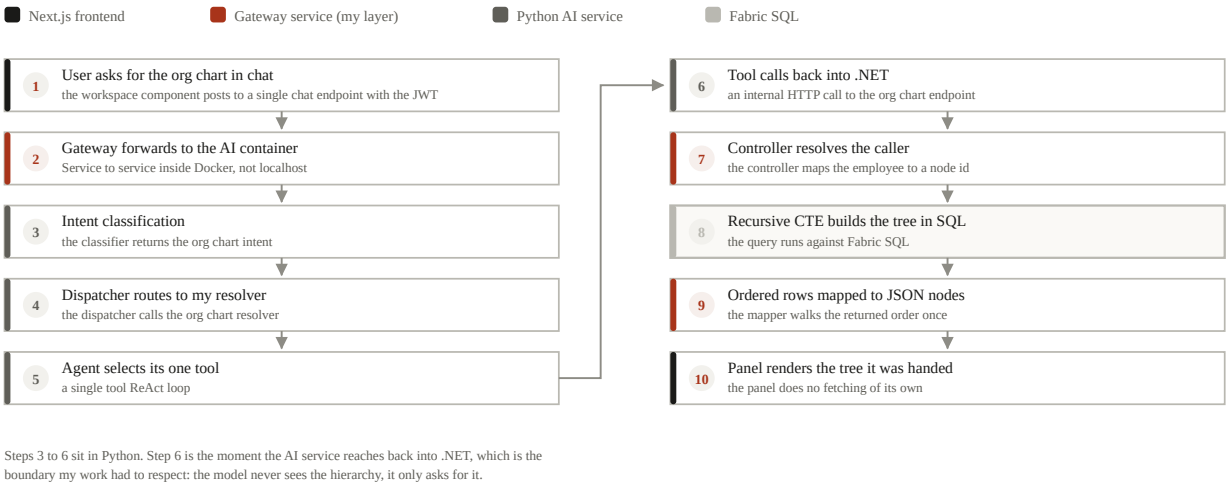
02

The system and my place in it

Applied. System tracing, Service boundaries, Docker Compose networking, LangChain tool design, JWT propagation

Before writing anything I had to understand how a question typed into the product actually reaches the database. This matters because the org chart is not a page the user navigates to. It is summoned conversationally: the user says "show me my org chart" to the AI assistant, the intent is classified, and a panel opens with the tree already populated.

The rule that governs the whole path is that the browser only ever calls one URL. Everything else happens behind that single call, invisible to the frontend. Tracing this end to end was my first task and it shaped every decision afterwards.



Steps 3 to 6 sit in Python. Step 6 is the moment the AI service reaches back into .NET, which is the boundary my work had to respect: the model never sees the hierarchy, it only asks for it.

Figure 1. Request path for a conversational org chart request across four services. Steps 7, 8 and 9 are mine.

Why the model is kept away from the data

The tool returns the tree as a structured artifact alongside the language model's summary text, rather than letting the model describe the hierarchy in prose. This is deliberate. A model that narrates a reporting line can invent one. Keeping the tree as data means the panel renders exactly what the database returned, and the model is limited to writing the sentence next to it.

03

Week by week

Week 1

Onboarding, and a bug in production

2 to 4 September

Three days. The first was spent learning the entire architecture and stack: RabbitMQ, Redis, Microsoft Fabric, T-SQL, Eventhouse RAG, Spark, Azure DevOps in place of GitHub, and a Docker Compose setup that brings up more than eight containers, one per service repository. By the end of day one I could also name what was broken: retrieval was naive and slow, prompts were bloated, and the speech pipeline barely filtered noise.

The ghost route defect

On day three the team was asked to test the admin side end to end. Working through it systematically, I found that the breadcrumb header rendered every path segment as a clickable link. In the Next.js App Router, some route folders exist only to group other routes and have no `page` file. Clicking one of those produced a 404 in production.
The fix was to stop rendering those segments at all, using the route grouping convention the lead developer had already established as the signal for which segments were organizational. I replaced the `.map` that built the breadcrumb array with a `.flatMap` returning an empty array for grouped segments.

I also considered rendering the segments as non clickable text instead of removing them, which is what most enterprise admin tools do, since a breadcrumb exists to tell the user where they are. I chose removal because introducing a disabled breadcrumb style in a hotfix meant adding a design token that did not exist. That tradeoff is worth stating plainly rather than pretending there was only one answer.

This shipped as my first pull request in Azure DevOps, a workflow I had not used before.

Applied. Next.js App Router, TypeScript, Array semantics, Systematic QA, Azure DevOps PRs, Docker Compose

What the fix actually turned on. Returning `null` from `.map` leaves a slot in the array that the separator logic still counts, which is what produced the doubled slashes in my first attempt. Returning an empty array from `.flatMap` removes the element entirely. The bug was fixed in the data the renderer consumes, not patched in the rendering. That distinction between filtering a structure and hiding its output is one I have used repeatedly since.

Week 2

Designing the feature, and building it in the wrong layer

7 to 11 September

The requirement from the 7 September meeting was specific: a user should see the people under them and above them, including budgeted

but unfilled positions, scoped to their corporation. The commitment was a working answer to "I want to see my org chart" by 18 September.

The first structural problem was that no reporting relationship existed in the data at all. Nothing recorded who worked under whom. So the first step was a Manager field on admin user creation, so that every employee record from that point forward carries a manager reference and the hierarchy becomes derivable rather than manually maintained.

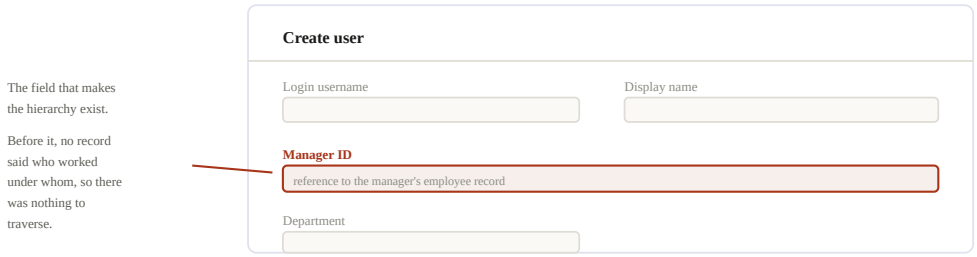


Figure 2. The manager reference added to admin user creation, shown here as a generic mockup. At this point the field was live in the interface but not yet wired, and cycle validation was flagged as a prerequisite before it could be.

I then agreed a schema with the database engineer and built the whole traversal layer in C#: an adjacency list keyed by manager, a recursive depth first descent, an upward chain of command walk, and a composition of the two into one tree for the frontend to render with a single recursive component.

That code worked. It was also in the wrong place, which I did not find out until the following week. Section 4 covers the reversal properly. What is worth recording here is the design reasoning, because that part survived the rewrite intact.

Choosing the structure

The access pattern is dominated by direct lookups: given a node, find its children; given a node, find its parent; repeat. I compared three representations against that pattern rather than against general graph theory.

The reasoning that mattered. The structure was not chosen because it is the textbook answer for graphs. It was chosen because the database hands back one flat result set, and the adjacency list is the structure that turns a flat set into a traversable graph in a single pass with no second query. The data boundary decided the structure.

Representation	*Lookup*	*Space*	*Verdict for this workload*
Adjacency matrix	O(1) edge test	O(V²)	Rejected. An org chart is a forest, so edges are roughly equal to nodes and the matrix is almost entirely zeros. Listing one manager's reports also means scanning a whole row.
Adjacency list hash map of manager to children	O(1) average	O(V + E)	Chosen. It materialises directly from a single flat result set in one pass, which matters because the database returns exactly that.
Materialised tree closure table or nested sets	Fastest reads	Expensive writes	Rejected for now. It is a schema commitment that is hard to reverse, and the schema was still moving weekly.

Details worth keeping

- **Cycle safety without relying on the database.** A foreign key on the manager column stops a dangling reference, but no foreign key can stop A reporting to B while B reports to A. I carried a visited set through the traversal for that reason, even though the constraint existed.
- **Two directions, two functions.** Descent and ascent were written separately and composed, so the frontend could later use either one alone without a rewrite, and each could be tested on its own.
- **Ancestors carry only the path child.** When focusing on a node, its ancestors include only the one child that lies on the path, not their other reports. That answers "where do I sit" rather than "what does the whole company look like", which was the stated requirement.
- **View state stays out of the domain.** The highlight flag marking the focused node is computed per request, not stored. It describes the view, not the organisation.

```
// the shape sent over the wire, in outline
class ChartNode {
    id, employee_id, title, department, status
    bool   is_focused        // view state, computed per request
    Manager  manager           // SHALLOW on purpose: no nested
reports,
                               // so serialisation cannot recurse
upward
    List<ChartNode> direct_reports
}
```

The comment explaining why the manager reference is shallow lives in the file next to it, not just in my head. Without it, the next person adds the reports collection back and discovers the problem at serialisation time.

A structural inconsistency found along the way

While adding my resolver to the Python dispatcher I found the file already contained two incompatible conventions for the same job. One intent called the Gateway directly over HTTP with a comment saying it existed to avoid model fabrication. Another skipped a resolver entirely and invoked its tool inline in the dispatcher. Neither used an agent loop, and my requirement matched neither.

Rather than copying whichever I saw first or quietly inventing a third silently, I chose the pattern that fit and documented in the function's own docstring that it is a deliberate project decision and not a copy of either existing convention, with a pointer to the fuller reasoning.

Applied. C# and .NET 10, Data structure selection, Recursive DFS, Cycle detection, DTO and API contract design, Schema design, Code documentation

What went into the docstring

That this path deliberately goes through a full agent loop rather than copying either existing convention; that the choice is a project decision and not an oversight; and a pointer to where the fuller reasoning lives. Plus a standing note that the data source behind it was still mocked at the time, and that the resolver itself would need no changes when the real source was swapped in.

The next person now meets three patterns instead of two, but the third one arrives with its reasoning attached. That is the difference between architectural drift and an architectural decision.

What I got wrong, stated plainly. I defended the application layer version because I had built it and because it was demonstrably correct. Correct and well placed are different properties. The lesson is that "it works" is the weakest possible argument for keeping a design, and being the author of something is a reason to be more suspicious of it rather than less.

Week 3

The reversal, and the first live chart

14 to 18 September

The lead developers reviewed the approach and made a call that changed the shape of the work: the traversal should not happen in the application layer at all. The database should return data in the shape the frontend needs, and the backend should map it rather than assemble it.

Applied. T-SQL, Recursive CTEs, Query design, Receiving architectural review, Refactoring across layers

My instinct was that this was a preference. It is not. The argument that convinced me is that every row the application layer pulls in order to discard it is a row that crossed the network for nothing. Building a focused view in C# means fetching the hierarchy and then throwing most of it away. Building it in SQL means the filtering happens where the data already sits, and only the needed rows move.

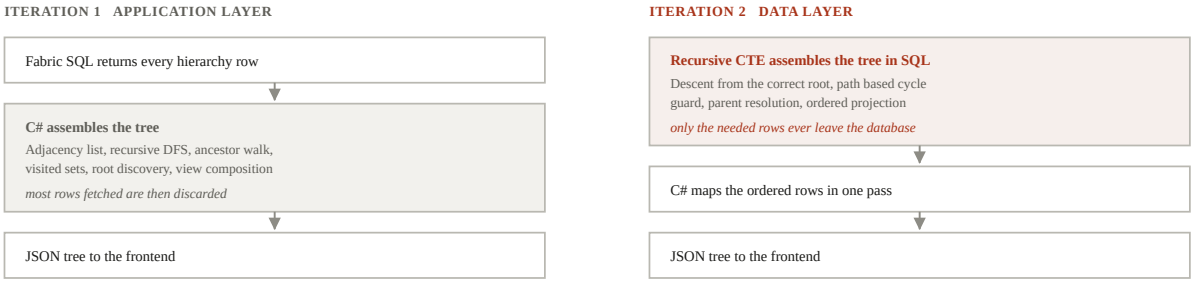


Figure 3. The same feature in two layers. Rewriting it was not wasted work: the traversal logic I had written by hand is what let me read and reason about the recursive query I then had to write.

First live rendering

By 15 September the panel was rendering the signed in user as a node, with their title, the people reporting to them and the person they report to, from real data rather than mocks. The commitment made on 7 September was for 18 September.

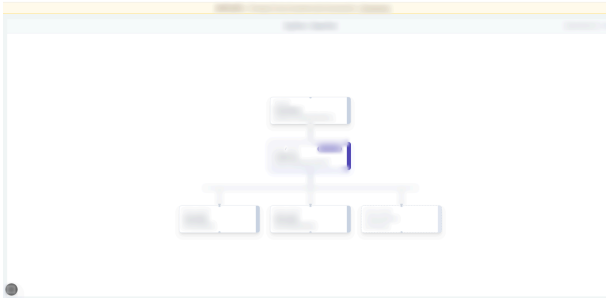


Figure 4. 10 September. Prototype route, mock data, structure proven before touching production.

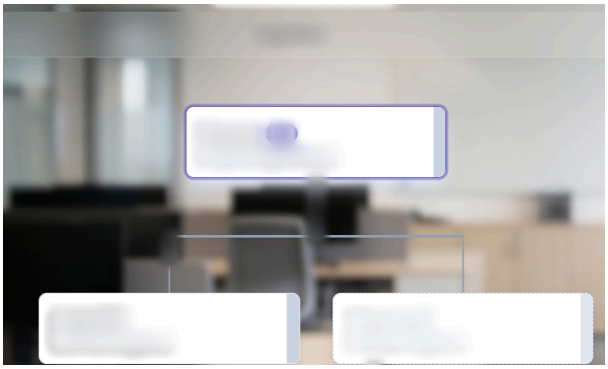


Figure 5. 15 September. In the product, live data, three days ahead of the committed date.

Week 4

Departments, a hybrid entity model, and infrastructure

21 to 24 September

The next requirement was a department view: not the focused person, but the entire department that person sits in, rendered from the head of department downward. This turned out to be a substantially harder problem than the first chart, because it introduced a second kind of node.

A department chart has to show departments themselves, since departments contain sub departments. So the tree contains two entity

types with different attributes, produced by two different recursive traversals, which then have to merge into a single structure the frontend can render with one component. Section 4 covers how that was done.

16 Sep **Scrollable chart shipped.** User's own chart rendering from live data with a scrollable graphic surface.

17 Sep **Schema simplification.** Dropped the position identifier column, renamed the manager reference to a parent node, and changed status into a type field. Populated parent department and manager references in the Departments table.

21 Sep **Zoom and scroll at scale.** Chart handling thirty seeded employees with zoom in and out, confirming the structure holds beyond a toy dataset.

22 Sep **Department query working.** Hybrid department and employee traversal rendering both entity types in one tree.

23 Sep **Permissions scoped.** Role based visibility agreed: top management sees the whole chart, a head of department sees only their own department including vacant positions.

24 Sep **Third query passing tests.** The combined query: ancestors and descendants, with department entities, in one traversal.

Applied. T-SQL migrations, Liquibase, Hybrid entity modelling, Azure Container Apps, Bicep and Terraform, CI/CD pipelines, Infrastructure as code, RBAC scoping

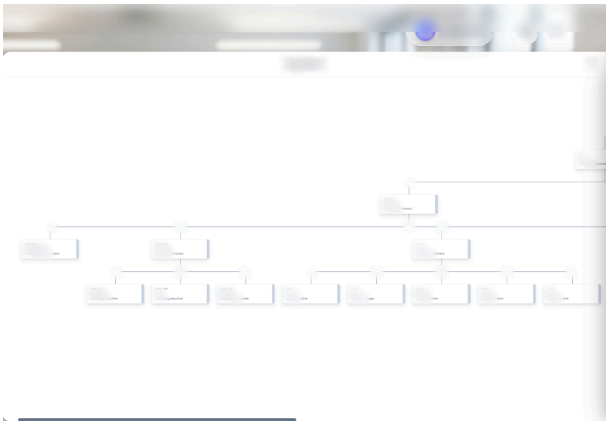


Figure 6. 21 September. Thirty employees, zoomable and scrollable.

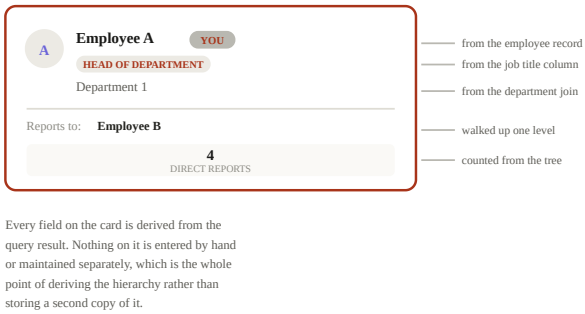


Figure 7. Node anatomy, drawn generically. Role, department, reporting line and direct report count are all derived rather than entered.

Database migrations

The department hierarchy needed a parent department reference that existed in the schema but had never been populated. I wrote the migration in T-SQL, registered it as a changelog, registered that changelog in the changelog index, and ran Liquibase against the Fabric SQL development database.

The migration is written to be safely re-runnable: it resolves identifiers by name rather than hardcoding them, raises explicit errors if the

expected departments or manager references are missing, and guards every insert with a check that the row does not already exist.

```
-- resolve identifiers by name, fail loudly if the world is not
as expected
SELECT @dept_id      = id FROM departments WHERE name =
'<department>';
SELECT @dept_head_id = manager_id FROM departments WHERE id =
@dept_id;

IF @dept_id IS NULL
    THROW 50001, N'Expected department not found.', 1;

IF @dept_head_id IS NULL
    THROW 50002, N'Department has no manager set.', 1;

-- every insert is guarded so the migration can be re-run safely
INSERT INTO org_nodes (employee_id, parent_node_id, department,
...)
SELECT @employee_id, @parent_node_id, '<department>', ...
WHERE NOT EXISTS (SELECT 1 FROM org_nodes WHERE employee_id =
@employee_id);
```

Incomplete data is a design constraint, not a blocker. The parent department relationships could not be fully populated because the real hierarchy was not fully known. Rather than waiting for it, the query falls back to inferring department structure from the manager chain: if the head of Finance reports to the head of IT, then Finance renders under IT even when the explicit parent link is missing. Building a system that degrades to a sensible answer when its data is partial turned out to be more valuable than a system that is correct only when the data is perfect.

Infrastructure side task

In parallel I was given a task in Azure DevOps: provision an isolated test environment from the infrastructure repository and verify the application runs in it. The process is to add an environment to the container app security profile, push to a branch, run the provisioning pipeline against that branch with a matching environment key and Fabric capacity, and then work through the results.

A full provision and security check run takes about an hour and a half before the containers are deployed and reachable. My run surfaced two environment specific defects, which are now recorded against that environment as reproducible findings rather than guesses.

Concept	*What I understood by the end of it*
Ingress	Public ingress decides which containers are reachable by URL at all. Services that exist only to be called by other services are given no public ingress, so no external request can address them, which is a meaningful part of the security posture rather than a deployment detail.
Replicas	Minimum and maximum replica counts per application. New instances spin up at a CPU threshold, and a second replica means one instance failing does not take the service down.
Bicep and Terraform	The deployment is declared rather than performed by hand. The environment definition lives in version control, which is also what makes an isolated test lab possible at all.
Isolated labs	A named environment provisions its own resource group and Fabric capacity, so testing does not touch development or acceptance environments.

04

Deep dive: three recursive queries

The substance of the backend work is three recursive common table expressions, each solving a different shape of the same problem. They were built in order, and each one exists because the previous one could not answer a new question.

No.	*Query*	*Question it answers*
1	**Focus view**	Where do I sit? Walk up the chain of command to the top, and down through every branch below me.
2	**Department view**	What does my department look like? Find my department, find its head, and render everyone beneath them, with departments and sub departments shown as entities in their own right.
3	**Combined view**	Both at once: ancestors and descendants of a focus node, with department entities interleaved in the correct positions.

Query 1: ascending and descending from a focus node

The descent is the standard recursive pattern. An anchor selects the starting node, and the recursive member joins each child to its already discovered parent, repeating until nothing new is found. The traversal records a path string as it goes.

```
-- generic shape of the descending traversal
WITH descendants AS (
    -- anchor: the node the view starts from
    SELECT n.id, CAST('/' + CAST(n.id AS varchar) + '/' AS
varchar(max)) AS visited_path
    FROM node_table n INNER JOIN start_node r ON r.id = n.id

    UNION ALL

    -- recursive member: join each child to its discovered parent
    SELECT c.id, CAST(p.visited_path + CAST(c.id AS varchar) + '/'
AS varchar(max))
    FROM node_table c
    INNER JOIN descendants p ON c.parent_node_id = p.id
)
```

Each row carries the path taken to reach it, built by appending its own identifier to its parent's path. That path is what the next section uses.

Cycle safety in SQL

The same problem as before appears in a different form. A recursive CTE that meets a cycle does not return a wrong answer, it runs until the recursion limit terminates it. Carrying the visited path as a delimited

string lets each step check whether a node is already on the path before following it.

```
-- only follow the edge if this child is not already on the path
WHERE CHARINDEX(
    '/' + CAST(c.id AS varchar) + '/',    -- delimiters matter
    p.visited_path
) = 0
```

Wrapping the identifier in delimiters before searching is what stops node 1 from matching inside node 12. That is the kind of detail that only shows up once the data is real, and it is a silent wrong answer rather than an error if you miss it.

Same idea, three implementations. Cycle protection appears three times in this work: a foreign key in the schema, a visited set in the C# traversal, and a path check inside the SQL. None of them is redundant, because each one catches a cycle at a different moment: at write time, at traversal time in the application, and at traversal time in the database. Understanding why the foreign key is not enough is what made the other two obviously necessary.

Query 2: the department view and its two entity types

The department chart starts from the focused person, resolves the department they belong to, then resolves the head of that department, and descends from the head. The descent always starts from the head of department rather than the focused person, because the view is about the department, not the individual.

The harder half is that departments are nodes too. A department can contain sub departments, so the tree has to interleave two entity types that share no attributes: a department has a name and a parent department but no employee identifier or job title, and an employee has the reverse.

The department traversal is a second, separate recursive expression following the same anchor and recursive member shape, joined on the parent department reference instead of the manager reference. It carries one extra column the employee traversal does not need: a depth counter, incremented on each recursive step, so the root department of the view can be distinguished from its descendants. Depth zero is the root, and that distinction is what the parent resolution in the next section keys off.

Merging two traversals into one tree

Two independent recursive traversals produce two sets of rows. The frontend needs one tree. Two problems had to be solved to get there.

Telling the two entity types apart

Department identifiers and employee identifiers come from different tables and can collide. Generating department node identifiers as negative numbers using a row number function guarantees they can never collide with employee identifiers, and makes the entity type readable from the identifier itself.

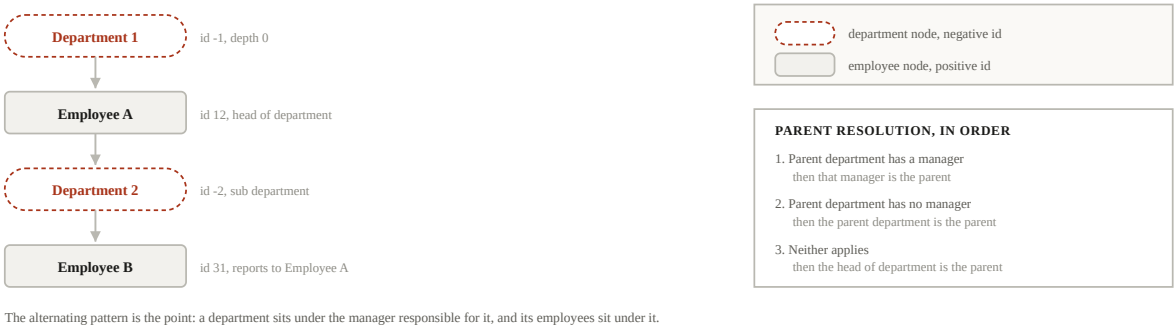


Figure 8. The hybrid node model. Sign encodes entity type, and a three case rule decides what each node hangs from.

Deciding what hangs from what

With two entity types in one tree, "who is this node's parent" stops being a single column lookup. A department might hang from the manager of its parent department, from the parent department itself if that department has no manager, or from the head of department at the top of the view. Employees can sit above a department, if they manage it, or below one.

This is resolved with an ordered fallback: take the first of the three candidates that exists. Ordering the cases correctly is what makes the alternating structure of departments and people come out right.

```
-- ordered fallback: first candidate that exists wins
CASE WHEN d.depth = 0 THEN NULL             -- root of the view has
no parent
     ELSE COALESCE(
         parent_dept_head.id,              -- 1. manager of the
parent department
         parent_dept_box.node_id,          -- 2. the parent
department itself
         head_of_dept.id                   -- 3. the head of
department
     )
END AS parent_node_id
```

The order of the three candidates is the whole design. Get it wrong and departments attach to the wrong level even though every individual case is expressible.

Applied. Recursive CTEs, Set based thinking, Polymorphic result sets, Cycle detection in SQL, Query optimisation, Fallback logic design

```
-- department branch: employee columns cast to null
SELECT d.node_id, <parent resolution> AS parent_node_id,
       CAST(NULL AS varchar) AS employee_id,
       d.name, CAST(NULL AS varchar) AS title,
       CAST(1 AS bit) AS is_department
FROM dept_nodes d ...

UNION ALL

-- employee branch: department columns cast to null
SELECT n.id, COALESCE(own_dept.node_id, n.parent_node_id) AS
parent_node_id,
       n.employee_id, e.display_name, n.title,
       CAST(0 AS bit) AS is_department
FROM descendants v INNER JOIN node_table n ON n.id = v.id ...
```

Both branches emit an identical column set, each casting nulls into the columns that do not apply to it, so two entity types arrive as one uniform stream of nodes and the frontend needs only one renderer.

Why the parent identifier has to be explicit. Combining two result sets gives rows in a defined order, but order alone carries no relationships. A consumer cannot infer from position which node belongs under which. The explicit numeric parent reference on every row is what turns an ordered list back into a tree, and it is what lets the frontend render both entity types with one recursive component instead of two.

Result

Both views are produced by the same rendering component on the frontend, because both queries return the same uniform node shape. Adding the department entity type did not require a second renderer.



Figure 9. The department view. Department entities (Finance, Management, Marketing, Sales) and employees resolved into one tree by a single query, with each person's reporting line and direct report count derived from the data.

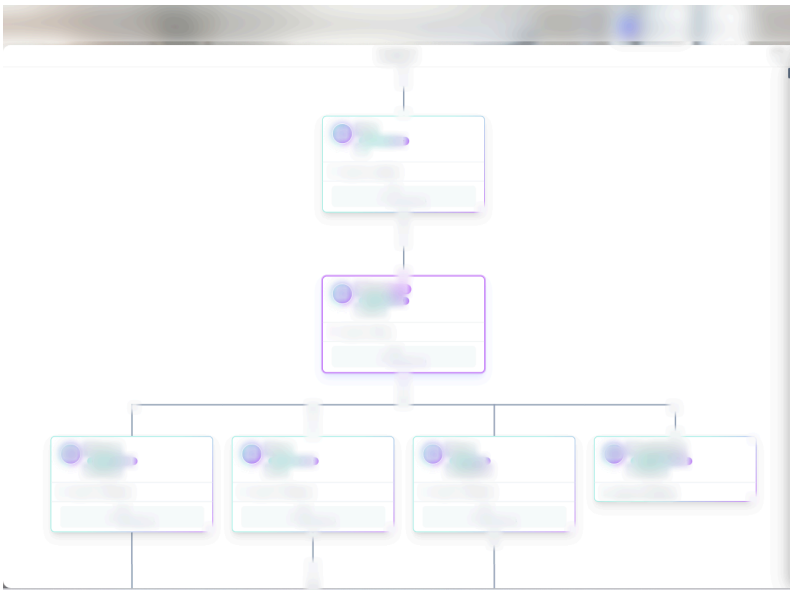


Figure 10. The focus view. Ancestors above the signed in user, direct reports below, with the user marked. The manager chain is walked upward from the focus node rather than filtered out of a full company tree.

05

Learning from other people

A significant part of what I learned in four weeks did not come from my own work. It came from watching how senior engineers diagnosed problems, and from two expensive mistakes that were not mine.

A latency fix worth studying

The AI assistant was taking around thirty five seconds to respond, degrading to outright timeouts. The cause was that the system retrieved the entire context on every single prompt, and the prompt management layer was not functioning.

The fix the lead developers implemented was a two layer approach. A first, small prompt carries only the list of available intents and classifies the request. A second, much larger prompt carrying full business logic and context is invoked only when the first layer is not confident.

The pattern underneath. A cheap operation decides whether the expensive one is needed. This is the same shape as a cache check, a bloom filter, or a query planner choosing an index. Recognising it as a general pattern rather than a one off trick is the part that transfers.

Two things I would want to verify rather than assume, if this becomes my work:

- **Self reported confidence is not calibrated.** The confidence score is produced by the language model itself and was not trained or validated against outcomes. A model can be confidently wrong. The threshold is a heuristic that happens to work, which is different from a measurement, and it should be described that way.
- **The classifier is now the bottleneck.** Intent classification alone was taking around six and a half seconds in the traces I read. For simple, unambiguous requests it is worth asking whether the language model needs to be in the path at all, which is what my current side task on a state machine alternative is exploring.

Two costly mistakes

Seeded test data with no expiry

A large batch of user accounts was seeded into a live database for testing and left running for a month. The accounts and their associated operations accrued cloud capacity charges the whole time before anyone noticed.

The failure was not the seeding. It was the absence of anything around it: no expiry on the test data, no scheduled teardown, no cost alert on the subscription, no separation between a workload that is being tested and one that is being billed, and no named owner responsible for cleaning it up. Each of those is cheap to put in place beforehand and expensive to discover afterwards.

What I took from it: test infrastructure needs an expiry date and an owner, or it quietly becomes production that nobody chose to pay for. When I began provisioning my own isolated lab environments a week later, this was directly on my mind.

On working inside someone else's architecture. The existing system has real problems: a message bus and cache that are installed but not integrated, naive retrieval, overloaded prompts. It is tempting to treat that as evidence about the people who built it. It is more accurate as evidence about what happens when deadlines are set without engineering input on a product already under sustained delivery pressure. Recording it as a systems failure rather than a personal one is both fairer and more useful, because only one of those framings suggests a fix.

Unbounded error logging

A logging path was found to be writing error rows without any bound, to the point where the volume became a stability concern for the system the logs existed to help diagnose.

What I took from it: anything that writes on failure needs a bound. Retention, rotation, sampling, or a separate store. Observability that can take down the system it observes has stopped being observability. The general shape is that any unbounded growth in a system is a defect even when each individual write is correct, which is the same reasoning as the seeded accounts arriving from a different direction.

Corrections to my own understanding

What I assumed	*What is actually true*
Redis in this system is a cache.	It is used for lock condition checking across transactions. Introducing caching would be a new use of Redis here rather than an extension of an existing pattern, which is a materially different thing to propose.
A message queue prevents data loss when input exceeds output.	A queue absorbs bursts. If arrivals exceed service rate on average rather than in spikes, the queue grows without bound and the loss arrives later with worse symptoms. Stability requires the consumer to keep up on average.
Proper architecture means routing requests through the message bus.	Queues are for work the caller does not wait on. A read that the interface needs in order to render is not that. The defensible use here is the fanout that follows a write, not the read itself.
The application layer is the natural place for traversal logic.	It is the natural place only when the data is already in memory. When the data lives in the database, filtering there means fewer rows cross the network, and the shape arrives ready to map.

06

How I worked, and how that changed

Problem solving approach

A pattern became visible across the four weeks that I had not consciously chosen:

1. **Find problems by testing systematically** rather than waiting for them to be reported. Both the breadcrumb defect and the dispatcher inconsistency came from working through something methodically rather than from a ticket.
2. **Trace a symptom to its structural cause** before fixing anything. The 404 was not a broken link, it was a renderer treating organizational routes as navigable ones.
3. **Look for existing structure before inventing new structure.** The route grouping convention already encoded which segments were organizational; I used that signal rather than writing my own list.
4. **Study a mature system that solved this at scale**, take what transfers, and discard what assumes resources I do not have. This is how a feature with years of precedent behind it got designed in under two weeks.
5. **Choose the smallest change that fully solves it**, so that it is cheap to review and introduces nothing new for the team to learn.

Working in an SME on a flagship project

Three things about this environment shaped how I worked, and none of them are technical.

Role boundaries are nominal. I am a backend intern, and in four weeks I have worked in TypeScript on the frontend, in C# in the gateway, in Python in the AI service, in T-SQL in the database, and in Bicep in the infrastructure repository. The useful response is to treat the job as whatever the feature needs rather than whatever the title says.

Sequencing is a political skill, not just a planning one. My first task was deliberately small, visible and low risk. Establishing that I could ship something safely into production came before proposing anything structural. I would not have described it that way at the time, but it is what happened, and it is why the architectural conversations in week three were possible.

Deadlines set without engineering input still have to be met or renegotiated. The commitment for 18 September was made in a room where the difficulty was not fully understood. Delivering it on 15 September was worth more than being right about it being ambitious.

Collaboration

Three engineers worked on one feature across four repositories with no merge conflicts. That was not luck. The boundaries were designed before the code was written: pure traversal functions in one file, orchestration in another, HTTP handling in a third, database and table design owned by the database engineer, rendering and the agentic path owned by the frontend engineer.

We also divided the work by what each of us wanted to get better at rather than by what each of us was already good at. I identified data structures and algorithms as my weakest area and took the component that was almost entirely that, while the frontend engineer took the rendering and interface work they wanted. Four weeks later the weak area is the part of the work I would most want to be questioned on.

Documentation as an engineering output

Two habits formed here that I intend to keep. First, when a decision differs from the surrounding code, the reason goes in the file next to it rather than in a message that disappears. Second, the limits I am aware of get written down explicitly. Noting that a query is unscoped and will need departmental scoping at larger headcounts is the difference between a limitation I chose and a limitation I missed, and only one of those is defensible.

Applied. C#, T-SQL, Python, TypeScript, ASP.NET Core, Microsoft Fabric SQL, Liquibase, LangChain, Next.js, Docker Compose, Azure DevOps, Azure Container Apps, Bicep, Terraform, RabbitMQ, Redis, Eventhouse RAG, Recursive CTEs, Graph traversal, Cycle detection, Schema migration, Infrastructure as code

---

On being reviewed. The most useful thing that happened to me in four weeks was being told that a working piece of code I had written belonged somewhere else. My first response was to defend it. The correct response, which I arrived at more slowly than I would like, was to ask what the reviewer could see that I could not. The reasoning turned out to be about data movement, not about style, and I could not have arrived at it alone.

The measurement I owe this project. Several claims in this journal are structural rather than numerical: the query moves fewer rows, the feature replaces a manual process, the latency work made the assistant usable. Each of those is currently an argument rather than a measurement. Capturing the baselines before the next change is the single highest value hour of work available to me, and it has a deadline, because a starting point that is not recorded before the change is gone permanently.

07

Skills review

Applied in production work during the period covered, with the specific evidence rather than a self assessment.

Technical

Skill	*Where applied*	*Evidence*
T-SQL, recursive CTEs	Org chart data layer	Three recursive query systems: focus view, department view, and combined hybrid view, all tested against seeded data.
Query design and optimisation	Fabric SQL	Moved tree construction out of the application layer so filtering happens where the data sits and fewer rows cross the network.
Database schema design	Hierarchy and department tables	Schema agreed with the database engineer, then simplified: position column dropped, manager reference renamed to parent node, status generalised to type.
Database migrations	Liquibase on Fabric SQL	Re-runnable T-SQL migration seeding a previously unused parent department field, with explicit failure conditions and existence guards.
C# and ASP.NET Core	Gateway service	Service, controller, DTO layer and orchestration; adjacency list and recursive DFS implementation in the first iteration.
Data structures and algorithms	Tree construction	Representation chosen by access pattern analysis; recursive DFS, ancestor walks, cycle detection implemented three ways at three layers.
API contract design	Org chart endpoint	Shallow manager reference preventing infinite upward serialisation; view state computed per request rather than stored.
TypeScript and Next.js	Admin dashboard	Production breadcrumb defect traced to App Router route grouping and fixed at the data layer.
Python, LangChain	AI service	Resolver and single tool agent integrated into an existing dispatcher, with the convention decision documented in place.
Infrastructure as code	Infrastructure repository	Provisioned isolated lab environments through the pipeline; container app ingress, replica policy, Bicep and Terraform definitions.
Azure DevOps and CI/CD	All repositories	Branching, pull requests, pipeline runs, and interpreting provisioning and security check results.
Docker Compose	Local environment	Eight plus containers, one per service repository, including service to service addressing.

Engineering judgement and working practice

Skill	*Demonstrated by*
System design	Tracing a request across four services before designing, then placing logic according to where data lives rather than where it is convenient to write.
Design justification	Comparing three representations against the real access pattern and being able to state why the rejected options were rejected.
Receiving review	Rebuilding a working feature in a different layer after an architectural challenge, and recording why the original reasoning was incomplete.
Root cause analysis	Tracing a 404 to a framework routing convention rather than patching the visible symptom.
Working in legacy and inconsistent code	Adding a third convention to a file that already held two incompatible ones, and documenting the decision so the inconsistency is navigable.
Designing for partial data	Falling back to manager chain inference when explicit department relationships are incomplete, so the feature degrades sensibly instead of failing.
Collaboration	Three engineers, four repositories, no merge conflicts, through boundaries designed before implementation began.
Scope negotiation	Delivering a commitment made without engineering input three days early, under sustained delivery pressure.
Deliberate skill development	Choosing the component built almost entirely on a self identified weak area rather than the one that played to existing strengths.

08

Open items and what comes next

Recording what is unfinished matters as much as recording what shipped. These are the things I know are outstanding, which is a different category from the things I have not thought of.

Known gaps in my own work

Item	*Status and reasoning*
Write time cycle prevention	The traversals are protected at read time in three places. Write time validation is the next piece of the same story: a check before the write that walks up from the proposed manager and rejects the change if it reaches the employee being edited, which is the traversal I have already written pointed at a candidate edge rather than an existing one.
Query scoping at scale	The current query is unscoped beyond the focus node's department. At several thousand employees across multiple tenants this needs scoping by corporation and department code. This is a limitation I have chosen for now rather than one I have missed, and it is noted in the code.
Caching	Hierarchy data is read constantly and changes rarely, which is the textbook case for a cache. The dependent piece is invalidation: changing one person's manager invalidates every cached subtree containing that node and every ancestor above it. That fanout is the work that genuinely belongs on a queue, because it must not block the user's save and must not be lost.
Node operations	Insert, delete and edit are not yet built. Editing is the hard one because moving a node changes hierarchy depth for everything beneath it and creates cycle risks that a simple insert does not.
Baselines before optimisation	Improvements cannot be stated as improvements without a recorded starting point. Tree build time, query latency at p50 and p95, and end to end response time need capturing before the next round of changes, not after.

Next

- Role scoped visibility: top management sees the full chart, a head of department sees only their own department including vacant positions.
- Viewing other people's charts, not only your own.
- Investigating a state machine as an alternative to language model intent classification, measured against the current approach rather than assumed to be faster.
- Hybrid retrieval for the assistant: adding lexical search alongside dense retrieval, which directly addresses the exact match failures that pure embedding search cannot handle.

Weeks one to four of a three and a half month internship running to December 2026. Screenshots are from the working product. Sections 1 to 4 describe work I owned; Section 5 records solutions designed by the lead development team, attributed as such. This is the external copy: employee names are

obscured, colleagues are referred to by role, internal identifiers are replaced with generic equivalents, and where a screenshot would have shown proprietary source the general form of the technique is written out instead. The engineering reasoning is intact without reproducing the employer's code or data.